

```

# =====
# Step 1: Import Required Libraries
# =====

import pandas as pd                # Used to work with datasets
from sklearn.model_selection import train_test_split  # Used to split
data into training and testing
from sklearn.preprocessing import LabelEncoder        # Converts text
into numbers
from sklearn.naive_bayes import CategoricalNB         # Naive Bayes
algorithm for categorical data
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# =====
# Step 2: Load Dataset
# =====

df = pd.read_csv("email_spam.csv")    # Load the CSV dataset
print(df.head())                     # Display first 5 rows

# =====
# Step 3: Convert Categorical Values into Numbers
# =====

encoder = LabelEncoder()              # Create Label Encoder object

df["Contains_Link"] = encoder.fit_transform(df["Contains_Link"])  #
Yes = 1, No = 0

df["Contains_Offer"] = encoder.fit_transform(df["Contains_Offer"]) #
Yes = 1, No = 0

df["Contains_Money_Words"] =
encoder.fit_transform(df["Contains_Money_Words"]) # Yes = 1, No = 0

df["Email_Length"] = encoder.fit_transform(df["Email_Length"])    #
Short = 0, Medium = 1

df["Spam"] = encoder.fit_transform(df["Spam"])    # Not Spam = 0, Spam
= 1

# =====
# Step 4: Select Features and Target
# =====

X = df[["Contains_Link",

```

```

        "Contains_Offer",
        "Contains_Money_Words",
        "Email_Length"]]
# Independent Variables (Input Features)

y = df["Spam"]
# Dependent Variable (Target)

# =====
# Step 5: Split Dataset
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,          # 20% data used for testing
    random_state=42         # Ensures same split every run
)

# =====
# Step 6: Create Naive Bayes Model
# =====

model = CategoricalNB()
# CategoricalNB is suitable because all features are categorical

# =====
# Step 7: Train the Model
# =====

model.fit(X_train, y_train)
# Learn patterns from training data

# =====
# Step 8: Make Predictions
# =====

y_pred = model.predict(X_test)
# Predict Spam or Not Spam for testing data

# =====
# Step 9: Evaluate Model
# =====

print("Accuracy:", accuracy_score(y_test, y_pred))
# Percentage of correct predictions

```

```
print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred))
# Shows correct and incorrect classifications

print("\nClassification Report")
print(classification_report(y_test, y_pred))
# Displays Precision, Recall, F1-score

# =====
# Step 10: Predict New Email
# =====

# Example:
# Contains_Link = Yes
# Contains_Offer = Yes
# Contains_Money_Words = Yes
# Email_Length = Short

new_email = [[1, 1, 1, 0]]

prediction = model.predict(new_email)

if prediction[0] == 1:
    print("\nPrediction: Spam Email")
else:
    print("\nPrediction: Not Spam")
```